AMAL JYOTHI
COLLEGE OF ENGINEERING
AUTONOMOUS
KANJIRAPPALLY

SYNOPSIS

AUREON: AN AI-POWERED SOFTWARE ENGINEERING INTELLIGENCE PLATFORM

Scrum Master

Rony Tom

Assistant Professor
Department of Computer Applications

DEPARTMENT OF
COMPUTER APPLICATIONS

1. INTRODUCTION

1.1 PROJECT OVERVIEW

Software development organizations often rely on multiple tools to manage projects, track development progress, monitor source code, and evaluate software quality. Using separate platforms for project management, version control, and code analysis can reduce productivity, limit project visibility, and make it difficult for teams to monitor software development effectively.

Aureon is a Software Engineering Intelligence Platform designed to provide a centralized environment for managing and monitoring software development activities throughout the Software Development Life Cycle (SDLC). The platform integrates project management, team collaboration, GitHub repository synchronization, and software quality analysis into a unified system, enabling organizations to efficiently manage software projects while maintaining better visibility into development progress and software quality.

The platform provides secure role-based access for Administrators, Project Managers, Team Leads, and Developers**,** ensuring that each user can access functionalities based on their responsibilities. Project Managers can create projects, manage teams, and monitor overall project progress, while Team Leads coordinate development activities and Developers manage assigned tasks and repository contributions. By integrating GitHub repositories, Aureon continuously synchronizes development activities, allowing users to monitor repository updates and project progress through interactive dashboards.

In addition, Aureon performs static code quality analysis to evaluate maintainability, code complexity, coding standards, and technical debt, helping development teams identify potential software quality issues during the development process. The platform also generates a comprehensive Project Health Score by analyzing project progress, repository activities, and software quality metrics, providing organizations with a centralized view of project status. Through role-based dashboards, reporting, and near real-time project monitoring, Aureon enables development teams to improve collaboration, monitor software quality, and make informed project management decisions within a single integrated platform.

1.2 PROJECT SPECIFICATION

The system consists of four actors. They are:

Administrator Functionalities

- **Secure Login:** Authenticate using authorized credentials to securely access the platform.
- **User Management:** Create, update, deactivate, and manage user accounts, roles, and access permissions.
- **Project Management:** Create, update, and manage software projects within the organization.
- **Team Management:** Create development teams, assign Project Managers and Team Leads, and manage team structures.

- **Platform Monitoring:** Monitor organization-wide projects, users, and system activities through the administrator dashboard.
- **System Configuration:** Configure platform settings, security policies, and repository integration settings.
- **Report Management:** Generate and access organization-level reports and project summaries.

Project Manager Functionalities

- **Secure Login:** Authenticate securely to access project management functionalities.
- **Project Management:** Create and manage software projects, define milestones, and configure project details.
- **Sprint Management:** Create sprints, define sprint duration, and monitor sprint progress.
- **Task Management:** Create high-level tasks and assign them to Team Leads.
- **Team Monitoring:** Monitor the progress and performance of individual teams through real-time dashboards.
- **Project Health Monitoring:** Monitor the Project Health Score, project status, repository activities, and software quality metrics.
- **Repository Monitoring:** View GitHub repository activities, commit history, and project development progress.
- **Dashboard & Analytics:** View project progress, sprint completion, team performance, and project analytics.
- **Report Management:** View approved team reports and generate project progress reports.

Team Lead Functionalities

- **Secure Login:** Authenticate securely to access assigned projects and team management features.
- **Task Distribution:** Divide project tasks into subtasks and assign them to developers.
- **Team Monitoring:** Monitor developer progress, task completion, and sprint activities.
- **Repository Monitoring:** Review team repository activities and developer contributions.
- **Software Quality Monitoring:** Monitor code quality metrics, maintainability, technical debt, and repository health.

- **Team Dashboard:** View team progress, software quality metrics, and Project Health Score related to the assigned team.
- **Report Approval:** Review, approve, and generate team performance reports before project evaluation.

Developer Functionalities

- **Secure Login:** Authenticate securely to access assigned projects and development tasks.
- **Task Management:** View assigned tasks, update task progress, and mark completed tasks.
- **GitHub Integration:** Push code changes and synchronize repository activities with the platform.
- **Project Dashboard:** View personal task status, sprint progress, and assigned project information.
- **Code Quality Analysis:** View software quality reports, code complexity analysis, and technical debt information for assigned modules.
- **Contribution Monitoring:** View personal contribution history and repository activities.
- **Personal Reports:** Access personal development progress and task completion reports.

2. REQUIREMENT GATHERING

2.1. Project Overview

Modern software development organizations rely on multiple independent tools for project management, source code management, repository monitoring, software quality analysis, sprint tracking, and reporting. Switching between these platforms often reduces productivity, limits project visibility, and makes it difficult for project managers to monitor overall project progress and software quality effectively.

The proposed project aims to develop Aureon, a centralized Software Engineering Intelligence Platform that integrates project management, team management, sprint management, GitHub repository synchronization, software quality analysis, and project health monitoring into a single web-based platform. The system provides role-based access for Administrators, Project Managers, Team Leads, and Developers, allowing each user to manage and monitor software development activities according to their responsibilities.

The platform continuously synchronizes repository activities, monitors project progress, evaluates software quality using static code analysis, and generates a comprehensive Project Health Score based on project and repository metrics. Interactive role-based dashboards provide near real-time visibility into project progress, team performance, repository activities, and software quality, enabling organizations to improve collaboration, monitor software development efficiently, and support informed project management decisions.

Objectives

- Develop a centralized Software Engineering Intelligence Platform.
- Implement secure user authentication and role-based access control.
- Manage software projects, teams, sprints, and development tasks.
- Integrate GitHub repositories for repository synchronization and activity monitoring.
- Perform static code quality analysis to evaluate maintainability and technical debt.
- Generate a comprehensive Project Health Score based on project and repository metrics.
- Develop role-based dashboards for project monitoring and visualization.
- Generate project, team, and software quality reports.
- Improve project visibility and team collaboration through a unified platform

2.2. System Scope

The proposed system is developed as an academic prototype for research and demonstration purposes. The mini project focuses on implementing the core functionalities of a Software Engineering Intelligence Platform without incorporating Artificial Intelligence. The system is designed to centralize software project management, team collaboration, repository monitoring, software quality evaluation, and project health monitoring within a single platform.

The scope of the system includes:

- Secure user authentication and role-based access control.
- Project creation and management.
- Team creation and management.
- Sprint creation and management.
- Task assignment and progress tracking.
- GitHub repository integration and synchronization.
- Repository activity monitoring.
- Static code quality analysis.
- Technical debt assessment.
- Project Health Score generation.
- Role-based dashboards for project monitoring.
- Team performance monitoring.
- Project and software quality report generation.
- Historical project and repository activity visualization.

The system is designed with a modular architecture that supports future enhancement and integration of Artificial Intelligence features such as predictive analytics, intelligent risk detection, software quality forecasting, developer assistance, and AI-powered recommendations.

2.3. Target Audience

The proposed platform is intended for organizations involved in software development and software engineering management.

The primary stakeholders include:

Administrator

Responsible for managing users, projects, teams, and overall system configuration. The Administrator oversees platform operations, controls user access, and monitors organization-wide activities.

Project Manager

Responsible for planning, organizing, and monitoring software projects. Project Managers create projects and sprints, assign tasks to Team Leads, monitor project progress, review project health, and generate project reports.

Team Lead

Responsible for coordinating the assigned development team. Team Leads distribute tasks among developers, monitor team progress, review software quality, approve team reports, and ensure timely completion of sprint activities.

Software Developer

Responsible for implementing assigned tasks, updating task progress, synchronizing source code with GitHub repositories, and monitoring personal code quality and development activities.

2.4. Modules

Module 1 – User Authentication & Role Management

Provides secure user authentication and role-based access control for Administrators, Project Managers, Team Leads, and Developers. The module manages user registration, login, profile management, and user permissions, ensuring secure and authorized access to the platform.

Module 2 – Project Management

Enables Project Managers to create, manage, and monitor software projects. The module supports project creation, project configuration, milestone management, and project lifecycle monitoring to ensure effective software project organization.

Module 3 – Team Management

Facilitates the creation and management of development teams. Project Managers can assign Team Leads to projects, while Team Leads coordinate developers and manage team activities to improve collaboration and productivity**.**

Module 4 – Sprint & Task Management

Supports sprint planning, task creation, task assignment, and progress tracking throughout the software development lifecycle. Project Managers assign tasks to Team Leads, who further distribute them among developers while monitoring completion status and sprint progress.

Module 5 – GitHub Repository Integration

Integrates GitHub repositories with the platform to synchronize repository activities, monitor commits, branches, pull requests, and contributor information, providing a centralized view of software development progress.

Module 6 – Software Quality Analysis

Performs static code quality analysis to evaluate maintainability, code complexity, coding standards, and technical debt using software quality analysis tools. The module helps development teams identify software quality issues and improve maintainability.

Module 7 – Project Health Monitoring

Continuously monitors software project progress by combining project management information, sprint activities, repository updates, and software quality metrics. The module generates a comprehensive Project Health Score to evaluate the overall health and progress of software projects.

Module 8 – Dashboard & Analytics

Provides interactive role-based dashboards for Administrators, Project Managers, Team Leads, and Developers. The dashboards present near real-time project progress, sprint status, repository activities, software quality metrics, team performance, and Project Health Score through graphical visualizations and analytics.

Module 9 – Report Generation

Generates project reports, team reports, software quality reports, repository activity reports, and Project Health reports. These reports assist Project Managers and Team Leads in monitoring project performance, evaluating software quality, and supporting project review and documentation.

2.5. User Roles

User Role	Responsibilities
Administrator	Manage user accounts, roles, projects, and teams; configure platform settings; monitor organization-wide activities; manage repository integrations; maintain system security; generate organization-level reports; and oversee overall platform administration.
Project Manager	Create and manage software projects, define sprints and milestones, assign tasks to Team Leads, monitor project progress, view Project Health Score, analyze team performance, access project dashboards, and generate project reports for decision-making.
Team Lead	Manage the assigned development team, distribute tasks among developers, monitor sprint progress, review repository activities, analyze software quality and technical debt, approve team reports, and monitor team performance through dashboards.
Developer	View assigned tasks, update task progress, synchronize code with GitHub repositories, monitor personal development activities, view code quality analysis, access personal dashboards, and review contribution and task completion reports.

2.6. System Ownership

The proposed system is owned and maintained by a Software Development Organization. The Administrator is responsible for managing the platform, including user accounts, roles, projects, teams, and system configurations. Project Managers, Team Leads, and Developers use the platform to manage software development activities, monitor project progress, evaluate software quality, and generate reports. The platform serves as a centralized environment for software project management and software engineering monitoring throughout the Software Development Life Cycle (SDLC).

2.7. Functional Requirements

- The system shall provide secure user authentication and role-based access control.
- The system shall allow the Administrator to manage users, roles, and access permissions.
- The system shall allow the Project Manager to create and manage software projects.
- The system shall allow the Project Manager to create and manage development teams.
- The system shall support sprint creation and task management.
- The system shall allow the Project Manager to assign tasks to Team Leads.
- The system shall allow Team Leads to distribute tasks among Developers.
- The system shall allow Developers to update task progress and completion status.
- The system shall integrate with GitHub repositories to synchronize development activities.
- The system shall monitor repository activities, including commits, branches, and contributor information.
- The system shall perform static code quality analysis to evaluate maintainability, code complexity, and technical debt.
- The system shall generate a comprehensive Project Health Score based on project and software quality metrics.
- The system shall provide role-based dashboards for Administrators, Project Managers, Team Leads, and Developers.
- The system shall display project progress, sprint status, team performance, repository activities, and software quality metrics.
- The system shall generate project, team, and software quality reports.
- The system shall maintain historical records of project activities and repository updates.

2.8. Industry / Domain

Industry

Software Development

Domain

• Software Engineering

• Project Management

- DevOps
- Software Quality Assurance
- Engineering Analytics
- Software Process Management

2.9. Data Collection Contacts

Name: Abisha Accamma Vinod

Organization: Confidential (Requested not to disclose)

Role: Senior Software Developer

Mode of Interaction: Personal Discussion

Contact no: 8078182384

2.10. Questionnaire for Data Collection

1. What are the biggest challenges your organization faces during the Software Development Life Cycle (SDLC)?

The senior developer explained that managing software projects becomes increasingly difficult as multiple teams work on the same project. Different tools are used for project management, source code management, issue tracking, and software quality analysis, making it difficult to obtain a unified view of project progress. Team members often switch between applications to access project information, which reduces productivity and increases management effort. Monitoring project health, identifying delays, and maintaining software quality across different teams also become challenging. A centralized platform that integrates these activities would significantly improve project visibility and management.

2. How is the software development team organized within your organization?

The organization follows a hierarchical software development structure. Every project is managed by a Project Manager who supervises multiple specialized teams. Each team is led by a Team Lead responsible for coordinating developers, distributing tasks, monitoring progress, and reviewing completed work. Developers focus on implementing assigned tasks while reporting their progress to the Team Lead. This structured workflow improves communication,

accountability, and coordination between different levels of project management while ensuring that software development activities remain organized throughout the SDLC.

3. How are software development tasks assigned and prioritized?

Project Managers initially assign high-level project requirements to Team Leads based on project objectives. Team Leads further divide these requirements into smaller development tasks and assign them to developers according to their technical expertise and workload. The organization also follows a priority-based task management system where tasks are categorized as High (Red), Medium (Orange), or Low (Yellow) priority. This prioritization helps developers focus on critical business requirements first while ensuring efficient workload distribution and timely project completion.

4. Which tools do you currently use for project management, repository management, and software quality analysis?

The organization currently uses internal project management software for planning and monitoring projects, Git repositories for version control and collaboration, and SonarQube for static code quality analysis. In addition, an internally developed AI assistant helps developers by providing code correction suggestions and improving development efficiency.

Although these tools perform their individual functions effectively, the information remains distributed across multiple platforms, making project monitoring and decision-making more time-consuming.

5. How do you currently evaluate the health of a software project?

Project health is assessed using several engineering metrics rather than relying on a single indicator. The organization considers sprint completion, task completion percentage, repository activities, pending issues, software quality reports, developer progress, and issue backlog while evaluating project status. Managers often combine information collected from different systems before making decisions. A single Project Health Score that summarizes these metrics would simplify project monitoring and improve management efficiency.

6. What information would you like to monitor from a centralized dashboard?

The senior developer suggested that a centralized dashboard should provide complete visibility into software development activities. Important information includes project progress, sprint status, pending tasks, repository activities, software quality metrics, Project Health Score,

developer contributions, team performance, engineering statistics, and project reports. Access to all these details from one interface would reduce the need to switch between multiple applications and help management make faster and more informed decisions.

7. How do you ensure software quality before code is merged into the main repository?

Before code is merged into the production branch, it undergoes several quality assurance processes. Static code analysis tools are used to identify code smells, maintainability issues, and potential bugs. Developers also perform peer code reviews, follow coding standards, and verify implementation through testing. Only after these quality checks are successfully completed is the code approved for merging. This process helps maintain software quality and reduces future maintenance efforts.

8. What challenges do Project Managers face while monitoring multiple development teams?

The senior developer stated that Project Managers often spend considerable time collecting project information from different teams and software tools. Since progress updates, repository activities, software quality reports, and engineering metrics are stored separately, obtaining a complete view of project status becomes difficult. Monitoring multiple teams simultaneously also increases management complexity. A centralized dashboard capable of integrating these activities would significantly improve project visibility and reduce management effort.

9. How is developer performance evaluated within your organization?

Developer performance is evaluated using multiple performance indicators rather than simply counting source code commits. The organization considers task completion, software quality, coding standards, collaboration with team members, problem-solving ability, repository contributions, and participation in sprint activities. This provides a more balanced and fair evaluation of each developer's overall contribution to the project while encouraging continuous learning and software quality improvement.

10. What improvements would you like to see in a future Software Engineering Intelligence Platform?

The senior developer recommended developing a centralized platform capable of integrating project management, repository monitoring, software quality analysis, Project Health monitoring, team performance evaluation, and automated reporting into a single system. Such a platform would improve collaboration, reduce manual management effort, provide better

project visibility, and support informed decision-making. The platform could also be extended in the future with Artificial Intelligence to provide predictive analytics, intelligent recommendations, and proactive risk identification for software development projects.

2.11. Geotagged Photos

3. FEASIBILITY STUDY

3.1. Introduction

A feasibility study is conducted to evaluate whether the proposed system can be successfully developed and implemented within the available technical, operational, and economic constraints. It helps determine the practicality of the project by analyzing the resources, technologies, development effort, implementation process, and expected benefits before the actual development begins.

For the proposed Aureon – Software Engineering Intelligence Platform**,** the feasibility study assesses the suitability of the selected technologies, the availability of development resources, the operational effectiveness of the platform, and the overall cost of implementation. Since the project utilizes open-source technologies and follows a modular development approach, it can be developed efficiently within the available academic schedule and resources. The feasibility study confirms that the proposed system is practical, reliable, and capable of supporting centralized software project management, repository monitoring, software quality analysis, and project health monitoring.

3.2. Objectives of the Feasibility Study

The objectives of the feasibility study are:

- To determine whether the proposed system satisfies the identified user and organizational requirements.
- To evaluate the technical resources required for developing the system.
- To determine whether the project can be completed within the available time and budget.
- To evaluate the operational usefulness and acceptance of the proposed system.
- To assess whether the system can be extended with advanced intelligent traffic management features in the future.

3.3. Information Assessment

The information collected during the requirement gathering process was carefully analyzed to identify the functional and operational requirements of the proposed system. Discussions with a senior software developer provided valuable insights into the software development workflow, project management practices, repository management, software quality evaluation, and team collaboration followed within a software development organization.

The assessment revealed that software development teams often rely on multiple independent tools for project management, repository monitoring, code quality analysis, and reporting. This fragmented approach makes it difficult for Project Managers and Team Leads to obtain a

centralized view of project progress, software quality, and overall project health. The discussion also highlighted the need for role-based project monitoring, efficient task management, centralized dashboards, automated reporting, and continuous monitoring of software quality throughout the Software Development Life Cycle (SDLC).

Based on these findings, the requirements for the Aureon – Software Engineering Intelligence Platform were identified. The system is designed to integrate project management, team management, sprint and task management, GitHub repository synchronization, software quality analysis, project health monitoring, role-based dashboards, and report generation into a unified platform. The information assessment confirms that the proposed system effectively addresses the identified challenges while providing a scalable foundation for future enhancements.The information collected indicates that an adaptive traffic signal management system would significantly improve traffic monitoring, decision-making, and operational efficiency compared to conventional fixed-time traffic signal systems.

3.4. Types of Feasibility

The proposed Aureon platform has been evaluated based on Technical Feasibility, Operational Feasibility, and Economic Feasibility.

Technical Feasibility

Technical feasibility evaluates whether the proposed system can be successfully developed using the available hardware, software, development tools, and technical expertise.

The proposed Aureon: Software Engineering Intelligence Platform is technically feasible because it utilizes well-established, open-source technologies and software engineering tools. The web application is developed using modern frontend and backend technologies, while MySQL is used for database management. GitHub APIs are integrated for repository synchronization, allowing the platform to retrieve repository information, commit history, pull requests, and developer activities.

For software quality analysis, the platform integrates industry-standard tools such as SonarQube, Pylint, and Radon. These tools perform static code analysis, identify software bugs, detect code smells, measure code complexity, evaluate maintainability, and generate software quality metrics. The collected metrics are processed and presented through centralized engineering dashboards and automated reports.

The required development tools, programming frameworks, APIs, and computing resources are readily available. Since the project is developed as an academic prototype, GitHub repositories and sample software projects can be used to demonstrate repository integration and software quality analysis without requiring access to proprietary industrial systems.

The project architecture is modular, enabling future integration of Artificial Intelligence features such as project delay prediction, intelligent code recommendations, developer assistance, and predictive engineering analytics without major architectural modifications.

Therefore, the proposed Aureon platform is technically feasible.

Operational Feasibility

Operational feasibility determines whether the proposed system effectively satisfies user requirements and improves existing software engineering processes.

During the requirement gathering phase, discussions with a senior software developer revealed that software organizations often use multiple independent tools for project management, version control, software quality analysis, and engineering reporting. Although these tools perform their individual tasks effectively, project managers frequently spend considerable time switching between applications to monitor project progress and prepare engineering reports.

The organization also follows a structured software development workflow where a Business Architect assigns tasks based on business priorities. Project Managers supervise multiple specialized teams, including Development, Testing, Quality Assurance, and DevOps teams. Task priorities are categorized into High, Medium, and Low priority levels with corresponding completion deadlines. Junior developers and fresh graduates initially receive training tasks before participating in production development.

The organization additionally utilizes an internally developed AI assistant to support developers by suggesting code corrections and improving development efficiency.

These observations directly influenced the design of Aureon. The platform provides centralized project management, Git repository integration, automated software quality analysis, engineering

dashboards, software health monitoring, report generation, and rule-based project risk notifications within a single environment.

By consolidating project information into a unified platform, Aureon reduces manual monitoring efforts, improves project visibility, simplifies engineering reporting, and enables project managers to make informed decisions based on real-time engineering metrics.

Therefore, the proposed Aureon platform is operationally feasible.

Economic Feasibility

Economic feasibility evaluates whether the expected benefits of the proposed system justify its development and maintenance costs.

The proposed Aureon platform is developed primarily using open-source technologies and freely available development tools. GitHub APIs, SonarQube Community Edition, Pylint, Radon,MySQL, Visual Studio Code, and other open-source frameworks significantly reduce software licensing costs.

Since the project is developed as an academic prototype, the primary investment consists of development effort, testing, and standard computing resources. No specialized hardware or expensive commercial software licenses are required during development.

The proposed platform has the potential to reduce manual project monitoring efforts, improve software quality through automated analysis, simplify engineering reporting, increase project visibility, and assist project managers in identifying project risks at an early stage. These improvements contribute to better software development practices and more efficient resource utilization.

Because the expected long-term benefits significantly outweigh the development cost, the proposed Aureon platform is considered economically feasible.

Estimated Development Resources Resource Cost

Python	Free		
React.js	Free		
MySQL Community Edition	Free	Visual Studio Code	Free
Git & GitHub	Free		

Therefore, the project is considered economically feasible.

3.5. Feasibility Conclusion

Based on the technical, operational, and economic feasibility analysis, the proposed Aureon – Software Engineering Intelligence Platform is feasible for implementation as an MCA Semester 3 Mini Project**.** The project utilizes reliable open-source technologies, integrates software project management with repository monitoring and software quality analysis, and addresses the practical challenges of managing software development activities through a centralized platform. The system can be successfully developed within the available academic schedule and resources.

The developed system will serve as a functional prototype capable of demonstrating centralized software project management, team collaboration, GitHub repository integration, software quality monitoring, Project Health evaluation, and role-based dashboards while establishing a strong foundation for future enhancements. In subsequent phases, the platform can be extended with Artificial Intelligence capabilities such as predictive project health analysis, developer performance analytics, intelligent risk detection, software quality forecasting, AI-powered recommendations, and advanced software engineering intelligence to support the development of a comprehensive AI-powered Software Engineering Intelligence Platform.

4. SYSTEM DESIGN

4.1 INTRODUCTION

The system design phase is a crucial stage in the development of Aureon: A Software Engineering Intelligence Platform for Project Health and Code Quality Monitoring. This phase converts the requirements identified during system analysis into a detailed technical design that serves as the foundation for system implementation. A well-designed system ensures reliability, maintainability, scalability, and an efficient user experience while supporting effective software project management.

The design process defines the overall architecture of the system, including the database structure, user interface, backend modules, and communication between different components. Aureon is designed to integrate project management functionalities with GitHub repository integration, code quality analysis, project health monitoring, task management, and report generation. The system follows a modular architecture, allowing each component to perform specific functions while working together as a complete software solution.

The platform provides role-based access for different users, including the Administrator, Project Manager, and Developer. Each user is provided with functionalities according to their responsibilities, enabling efficient project administration, team collaboration, repository management, and project monitoring.

Special emphasis has been placed on designing a responsive user interface, secure authentication mechanism, efficient database management, and seamless interaction between system modules. The proposed system design ensures that Aureon provides an organized environment for monitoring software projects, evaluating project health through rule-based metrics, and improving overall project management efficiency. Furthermore, the modular design provides a strong foundation for integrating advanced intelligent features in future versions of the system.

4.2 UML DIAGRAM

The Unified Modeling Language (UML**)** is a standardized visual modeling language used to analyze, design, and document software systems. Developed by the Object Management Group (OMG), UML provides a common graphical notation that helps developers, designers, and stakeholders understand the structure and behavior of a software application. Unlike programming languages such as Python, Java, or C++, UML is not used to develop software

directly; instead, it provides graphical representations of system components and their interactions.

In the development of Aureon, UML diagrams are used to represent the structural and behavioral aspects of the system. They provide a clear understanding of user interactions, system processes, data flow, and communication between various software components. These diagrams assist developers in understanding the system architecture, validating requirements, simplifying implementation, and facilitating future maintenance.

The UML diagrams prepared for the Aureon system include:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- Deployment Diagram
- Component Diagram

4.2.1 Use Case Diagram

A Use Case Diagram is a behavioral UML diagram that illustrates the interactions between users and a software system. It provides a high-level representation of the functional requirements by identifying the different actors and the operations they can perform. The diagram helps define the system boundary while showing how each actor interacts with the available system functionalities.

For the Aureon system, the primary actors are the Administrator, Project Manager, and Developer. The Administrator manages user accounts, projects, and system configurations. The Project Manager is responsible for creating and managing projects, assigning tasks, monitoring project progress, integrating GitHub repositories, viewing project health reports, and generating reports. Developers can access assigned projects, update task status, connect repositories, view code quality metrics, and monitor their assigned work.

The use case diagram provides a clear overview of the major functionalities of the Aureon platform, including user authentication, project management, team management, task management, GitHub repository integration, static code quality analysis, project health

monitoring, dashboard visualization, and report generation. It serves as an important reference during system development by ensuring that all functional requirements are properly represented while maintaining clear communication among developers, stakeholders, and end users.

Figure 1 : Use case diagram forAureon

4.2.2 Sequence Diagram

A Sequence Diagram is a behavioral UML diagram that illustrates the chronological order of interactions between users and different components of a system. It represents how messages are exchanged among objects to accomplish a particular function or process. The diagram helps visualize the sequence of operations performed by the system in response to user actions. The primary elements of a sequence diagram include actors, lifelines, activation bars, messages, and return messages, which collectively describe the flow of communication within the system.

In the Aureon system, sequence diagrams are used to represent the interaction between users and system modules during various operations such as user authentication, project creation, task assignment, GitHub repository integration, code quality analysis, project health evaluation, and report generation. These diagrams provide a clear understanding of how requests are processed, how data is exchanged between system components, and how responses are returned to the users.

Sequence diagrams play an important role in analyzing system behavior, validating functional requirements, identifying communication between modules, and ensuring that system processes are executed in the correct order. They also serve as a valuable reference during implementation, testing, and future system maintenance.

Uses of Sequence Diagram

- Models and visualizes the flow of interactions between users and system components.
- Illustrates the detailed execution of system functionalities such as project management and repository analysis.
- Helps understand the communication between different modules of the Aureon platform.
- Represents the sequence of messages exchanged during various system operations.
- Assists developers in analyzing system behavior, validating requirements, and identifying design improvements.
- Serves as a reference for system implementation, debugging, testing, and maintenance.

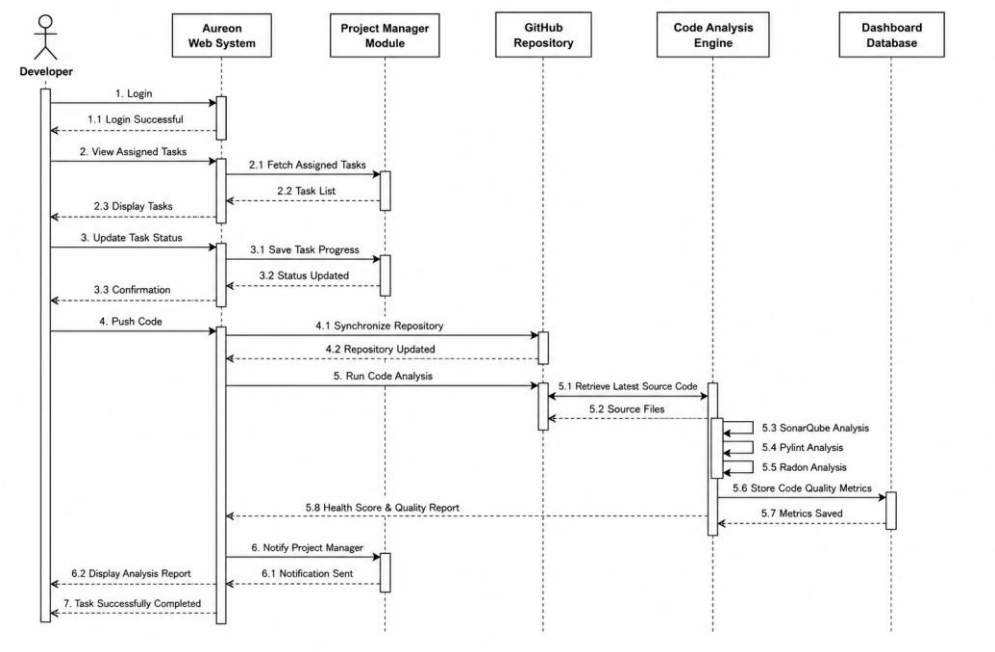

Figure 2 : Sequence diagram for Aureon

4.2.3 State Chart Diagram

A State Chart Diagram is a behavioral UML diagram that illustrates the different states of a system or an object and the transitions that occur between those states based on specific events

or conditions. It provides a visual representation of the dynamic behavior of the system throughout its lifecycle. The main elements of a state chart diagram include the Initial State, State, Transition, Event, Action, and Final State, which together describe how the system responds to user actions and internal processes.

In the Aureon system, the State Chart Diagram is used to represent the various states involved in managing software projects and user activities. It illustrates how the system transitions through different stages such as user login, project creation, task assignment, repository integration, code quality analysis, project health evaluation, report generation, and user logout. The diagram provides a clear understanding of how the system responds to different events while ensuring that each process follows the correct sequence.

State Chart Diagrams are valuable for analyzing the behavior of the Aureon platform, validating system workflows, and identifying possible state transitions during project execution. They also help developers understand the lifecycle of different system processes, making implementation, testing, and maintenance more effective.

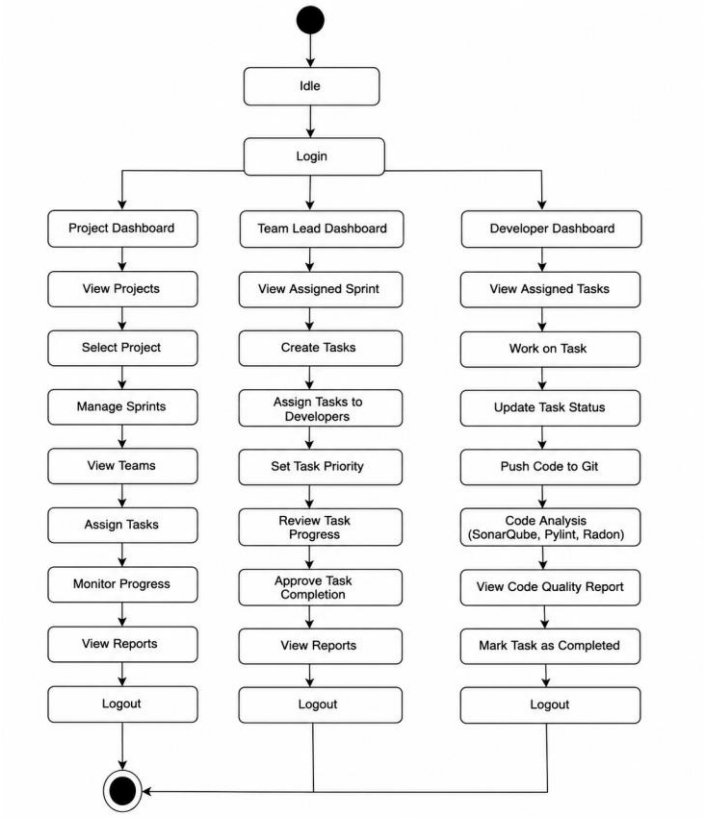

Figure 3 : State Chart diagram for Aureon

4.2.4 Activity Diagram

An Activity Diagram is a behavioral UML diagram that represents the workflow of a system by illustrating the sequence of activities performed to complete a specific process. It shows how one activity leads to another through different control flows, including sequential, parallel, and conditional execution. Activity diagrams help visualize the operational flow of a system from the starting point to the completion of a process, making them useful for analyzing business processes and software workflows.

In the Aureon system, the Activity Diagram is used to represent the workflow of various system operations such as user authentication, project creation, team management, task assignment, GitHub repository integration, code quality analysis, project health evaluation, and report generation. It illustrates how users interact with the system, how decisions are made during different stages, and how the system processes requests before producing the desired output. The diagram provides a clear understanding of the overall workflow and helps ensure that each process is executed efficiently and in the correct sequence.

The key components of an Activity Diagram are:

- **Initial Node:** The starting point of the workflow, represented by a filled black circle.
- **Activity:** A task or operation performed within the system, represented by a rectangle with rounded corners.
- **Control Flow:** Represents the sequence of execution between activities, shown using directional arrows.
- **Decision Node:** Indicates a point where the workflow branches based on a condition, represented by a diamond.
- **Merge Node:** Combines multiple alternative paths into a single flow after a decision point.
- **Fork Node:** Splits a process into multiple parallel activities that can execute simultaneously.
- **Join Node:** Synchronizes parallel activities into a single workflow before continuing.
- **Final Node:** Represents the completion or termination of the workflow, shown as a filled black circle enclosed within another circle.
- **Object Flow:** Represents the movement of data or objects between different activities during system execution.

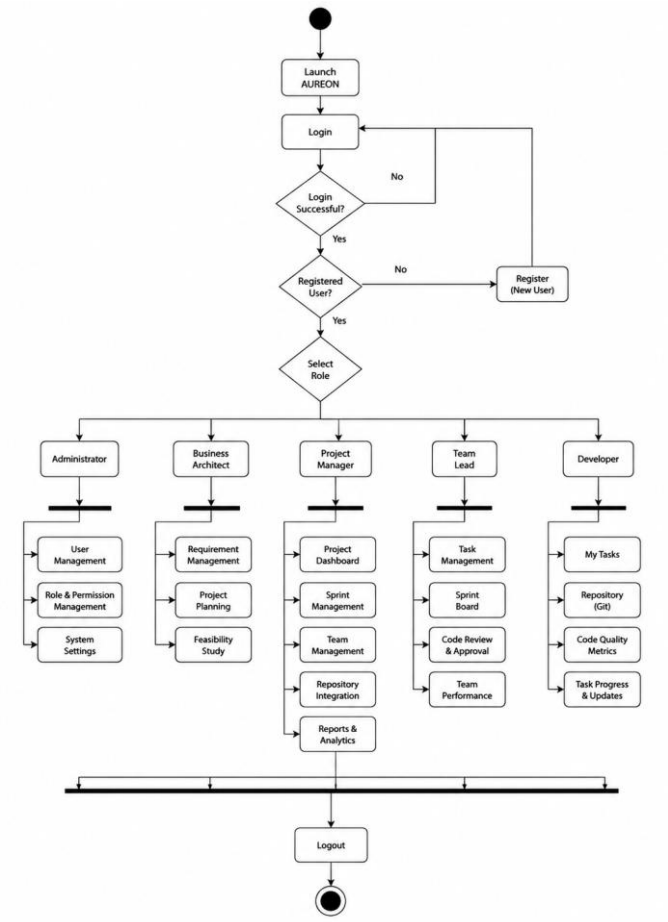

Figure 4 : Activity diagram for Aureon

4.2.5 Class Diagram

A **Class Diagram** is one of the most important structural UML diagrams used in object-oriented system design. It provides a static view of the system by illustrating its classes, attributes, methods, and the relationships between them. Class diagrams serve as a blueprint for the software architecture by defining how different entities within the system are organized and how they interact with one another. They help developers understand the overall structure of the application before implementation and support effective system design and maintenance.

In the **Aureon** system, the Class Diagram represents the core entities involved in project management and code quality monitoring. The major classes include **User**, **Administrator**, **Project Manager**, **Developer**, **Project**, **Task**, **Repository**, **Code Analysis**, **Project Health**, and **Report**. These classes are interconnected through various relationships such as associations, inheritance, and dependencies, enabling efficient management of projects, team members,

repositories, tasks, and project health evaluation. The diagram provides a clear understanding of the system's internal structure and supports the development of a modular and maintainable application.

Important Components of a Class Diagram:

- **Class:** Represents a blueprint for objects and contains attributes and methods that define the properties and behavior of system entities.
- **Interface:** Defines a set of abstract methods that can be implemented by different classes to achieve common functionality.
- **Object:** An instance of a class that represents a real-time entity within the system.
- **Association:** Represents a relationship between two classes that interact with each other.
- **Aggregation:** Represents a weak whole-part relationship where one class contains another, but both can exist independently.
- **Composition:** Represents a strong whole-part relationship where the lifecycle of one class depends on another.
- **Inheritance:** Represents an "is-a" relationship in which a subclass inherits the attributes and methods of its superclass.
- **Dependency:** Represents a relationship where one class depends on another class to perform a specific operation.

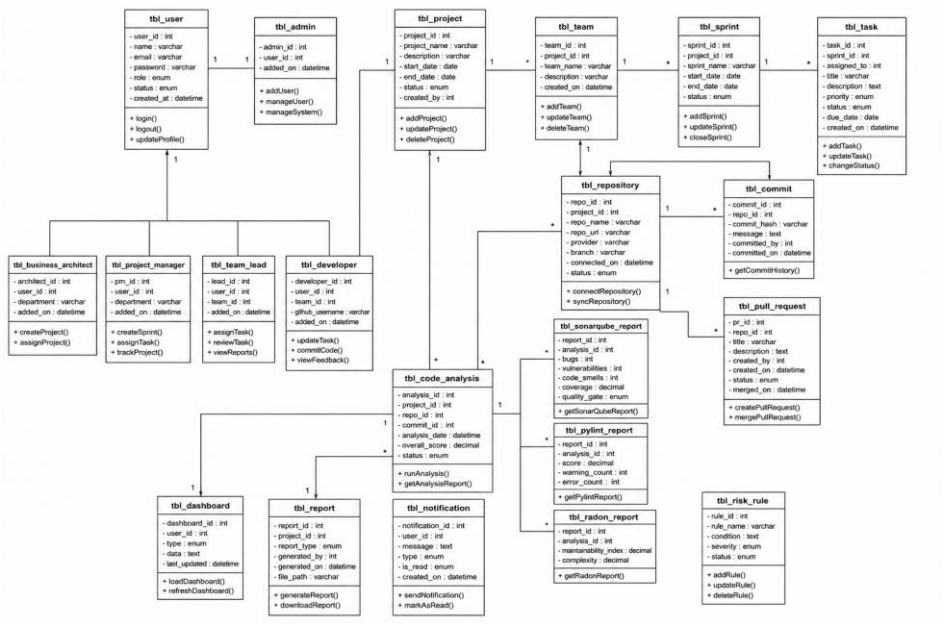

Figure 5 : Class diagram for Aureon

4.2.6 Object Diagram

An Object Diagram is a structural UML diagram that represents a snapshot of a system at a specific point in time. It illustrates the instances of classes, known as objects, along with the relationships between them. Object diagrams are derived from class diagrams and provide a practical view of how objects interact during the execution of the system. While class diagrams describe the static structure of the system, object diagrams demonstrate how the defined classes are instantiated and connected in real-world scenarios.

In the Aureon system, the Object Diagram represents instances of the major system entities such as Administrator, Project Manager, Developer, Project, Task, Repository, Code Analysis, Project Health, and Report. It illustrates how these objects are connected while performing operations such as project creation, task assignment, repository integration, code quality analysis, project health monitoring, and report generation. The diagram helps visualize the actual relationships between objects during system execution and provides a better understanding of the system's runtime behavior.

Key Components of an Object Diagram:

- **Object:** An instance of a class representing a specific entity within the Aureon system, such as a project, user, or repository.
- **Class:** Defines the structure and behavior from which objects are created, including their attributes and operations.
- **Link:** Represents the relationship or connection between two or more objects.
- **Attribute:** Describes the properties or characteristics of an object using name-value pairs.
- **Value:** Represents the actual data assigned to an object's attribute at a particular point in time.
- **Operation:** Represents the actions or functions that an object can perform based on its associated class.
- **Multiplicity:** Specifies the number of object instances that can participate in a relationship

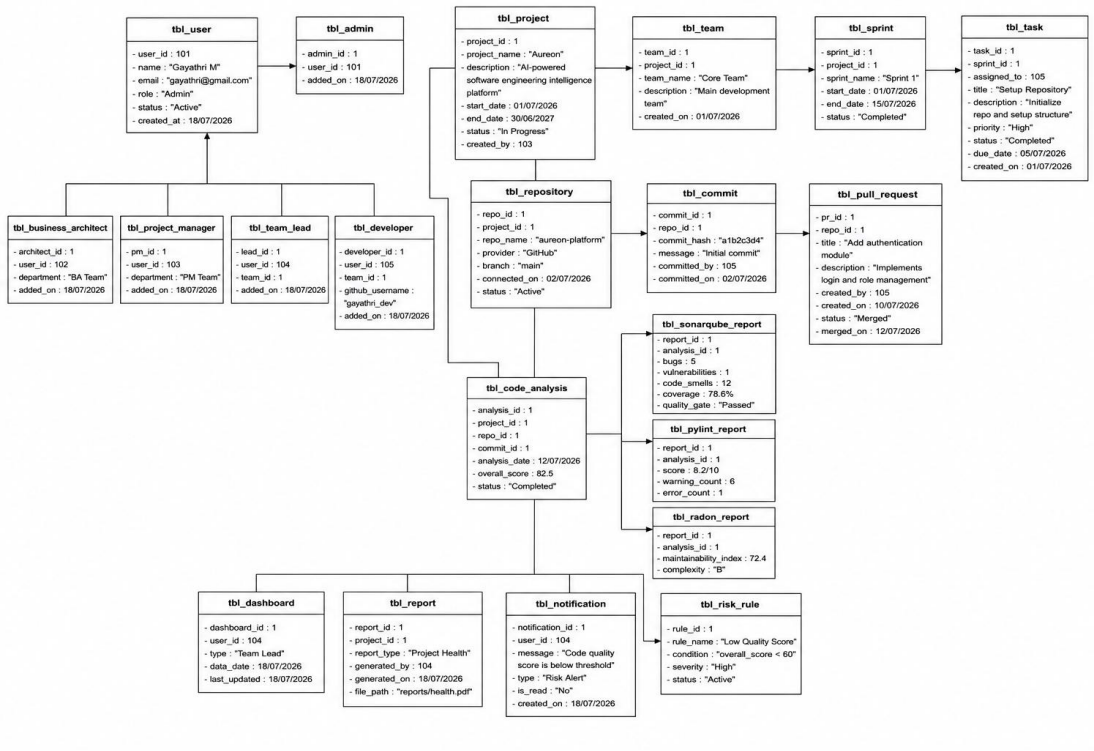

Figure 6 : Object diagram for Aureon

4.2.7 Component Diagram

A Component Diagram is a structural UML diagram that illustrates the organization of a software system into independent components and shows how these components interact to provide the overall functionality of the application. It provides a high-level view of the system architecture by representing the major software modules and the dependencies between them. Component diagrams help developers understand the modular structure of the system, simplify software design, and support efficient development, testing, and maintenance.

In the Aureon system, the Component Diagram represents the major functional modules, including the User Management, Authentication, Project Management, Task Management, GitHub Repository Integration, Code Quality Analysis, Project Health Monitoring, Report Generation, and Database components. Each component performs a specific responsibility while communicating with other components to ensure smooth system operation. This modular architecture improves maintainability, enhances scalability, and allows individual components to be updated or modified without affecting the overall system.

Key Components of a Component Diagram:

- **Component:** A modular software unit that provides a specific functionality within the Aureon system.
- **Interface:** Defines the services or operations that a component provides or requires to interact with other components.
- **Port:** A communication point through which a component exchanges information with other components.
- **Connector:** Represents the communication link between two or more components.

- **Dependency:** Indicates that one component relies on the services or functionality provided by another component.
- **Association:** Represents the interaction or connection between different components within the system.

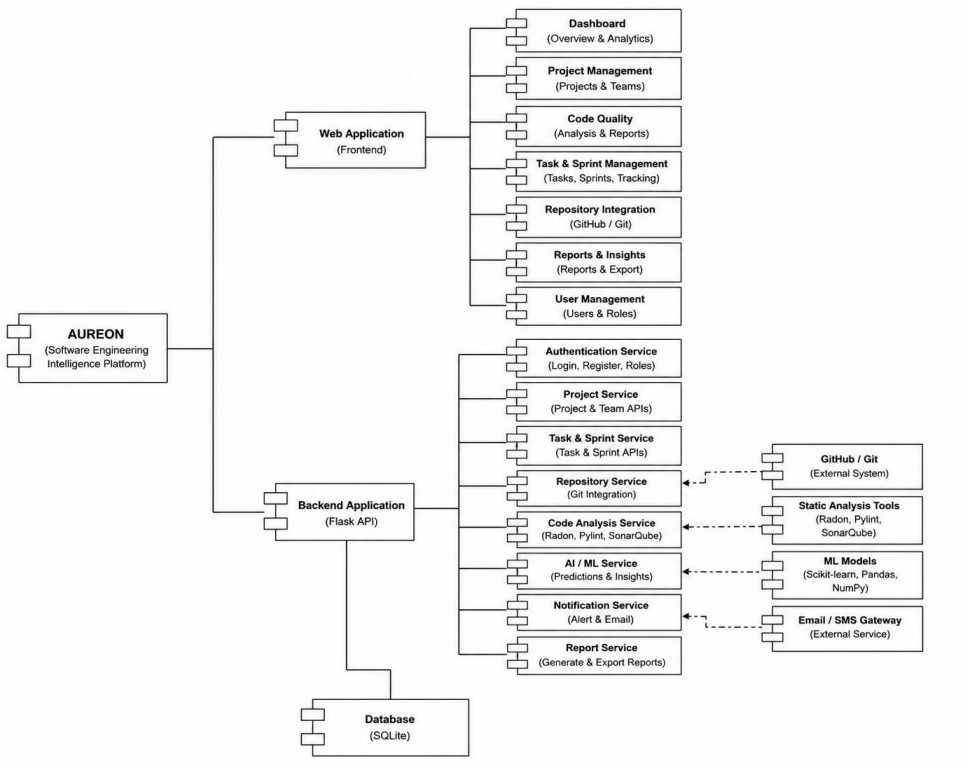

Figure 7 : Component diagram for Aureon

4.2.8 Deployment Diagram

A Deployment Diagram is a structural UML diagram that illustrates the physical deployment of a software system by showing the hardware environment, software components, and the communication between different nodes. It provides a static view of how the application is deployed and executed on physical or virtual devices. Unlike other UML diagrams that focus on the logical structure of a system, the deployment diagram emphasizes the hardware architecture and the distribution of software components across different nodes.

In the Aureon system, the Deployment Diagram represents the physical architecture of the application, including the Client Device, Web Server, Application Server, GitHub Repository, and Database Server. Users access the system through a web browser on the client device, where requests are sent to the application server. The application server processes user requests, manages project and task operations, performs code quality

analysis, communicates with connected GitHub repositories, and stores system data in the database. The communication between these nodes ensures efficient data processing, secure access, and smooth system operation.

Key Components of a Deployment Diagram:

- **Node:** A physical or virtual device that hosts software components, such as the client device, application server, or database server.
- **Component:** A software module deployed on a node to perform specific functionalities within the Aureon system.
- **Artifact:** A deployable software file or resource, such as the web application, configuration files, or database scripts.
- **Deployment Specification:** Defines the configuration and deployment details required for executing software components on a particular node.
- **Association:** Represents the relationship between nodes and the components deployed on them.
- **Communication Path:** Represents the network connection through which nodes exchange data and communicate with each other.

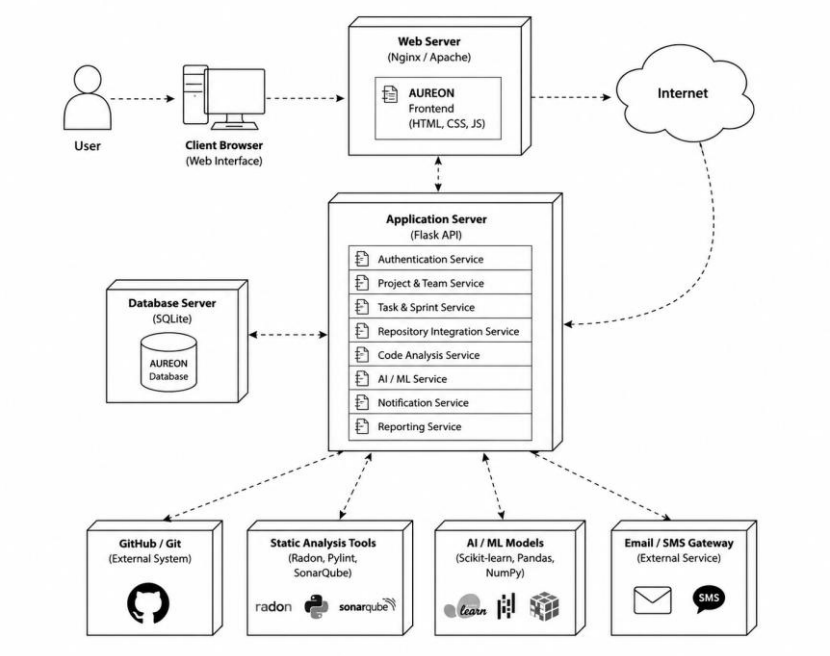

Figure 8 : Deployment diagram for Aureon